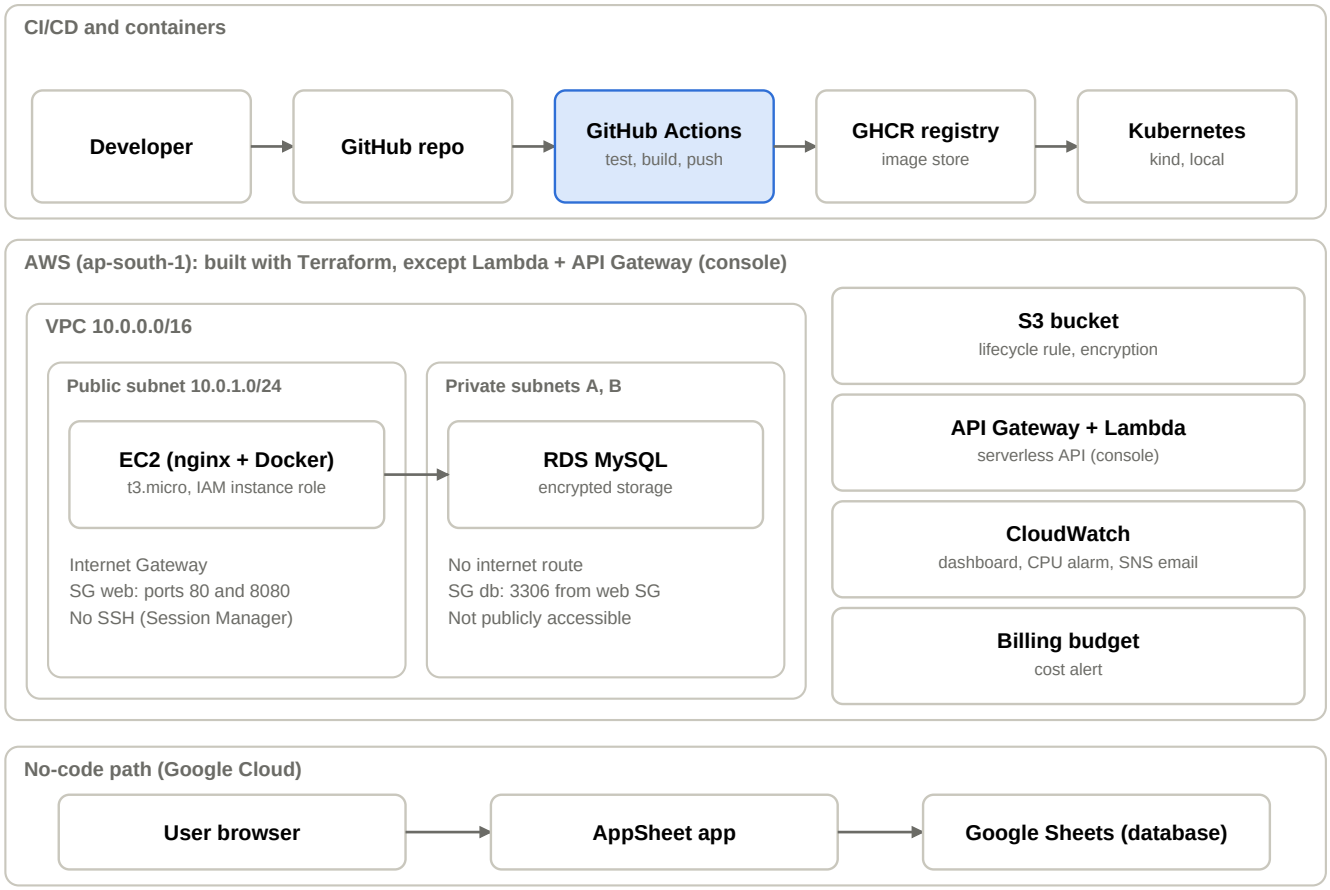
CloudOps Enterprise Platform: Architecture Document

Submitted by: Smarika Choudhary
Cloud: AWS (Mumbai, ap-south-1), plus GitHub, Kubernetes (kind) and Google AppSheet
GitHub repository: https://github.com/smarikachoudhary/cloudops-capstone
Screenshots (in order):
https://github.com/smarikachoudhary/cloudops-capstone/blob/main/SCREENSHOTS.md

1. Project Summary

This project builds a cloud-native application ecosystem. The AWS infrastructure is defined with Terraform (Infrastructure as Code). Containerised microservices run on Kubernetes. A GitHub Actions pipeline tests, builds and publishes the Docker image. A serverless API, monitoring, a no-code app and a big data and ML design complete the platform. To control cost, the AWS resources were created, screenshots were captured, and everything was then removed: the Terraform-built resources with *terraform destroy* (27 resources), and the Lambda function and API Gateway, which were created from the AWS console, manually.

2. System Design (Architecture Diagram)



system architecture: CI/CD, AWS and no-code layers

A push to GitHub triggers the pipeline, which publishes the image that Kubernetes pulls. The VPC, EC2, RDS, S3, IAM and CloudWatch layers are created by Terraform; the serverless API (Lambda and API Gateway) was created from the console.

3. Module-wise Implementation

Screenshot numbers refer to the file SCREENSHOTS.md in the GitHub repository.

Module	What was built	Evidence (screenshots)
Infrastructure	VPC, public and private subnets, Internet Gateway, Security Groups, EC2 running nginx	1, 4, 6, 7
Docker vs VM	nginx run directly on the EC2 VM (port 80) and in a Docker container (port 8080)	2, 3, 5
Storage and DB	S3 bucket with lifecycle policy and encryption, encrypted RDS MySQL in private subnets	8, 9, 10, 11
Serverless	Lambda function cloudops-hello-api triggered by API Gateway (GET /hello), created from the console	12, 13, 14, 15
Security	IAM role and instance profile for EC2, read-only IAM user, encryption on S3, RDS and the EC2 disk	22, 9, 11
Monitoring	CloudWatch dashboard, CPU alarm with SNS email, billing budget	21, 23, 24, 25
Kubernetes	frontend, backend and cicd-app deployments with 2 replicas each	16, 17, 18, 19, 20
CI/CD	GitHub Actions: test, build and push image to GHCR, validate manifest	26, 27
No-code app	Google AppSheet app connected to Google Sheets (Mendix needed a work email)	28, 29, 30, 31
Cleanup	terraform destroy (27 resources); Lambda function deleted from the console	terraform destroy output

4. VM Setup

An EC2 instance (t3.micro, Amazon Linux 2023) is launched in the public subnet with an IAM instance profile (AmazonSSMManagedInstanceCore) and an encrypted root volume. User data installs nginx and Docker. nginx serves the web page on port 80, and a Docker nginx container runs on port 8080. The web security group allows only HTTP (80) and the Docker page (8080). There is no SSH rule: the server is administered through AWS Systems Manager Session Manager.

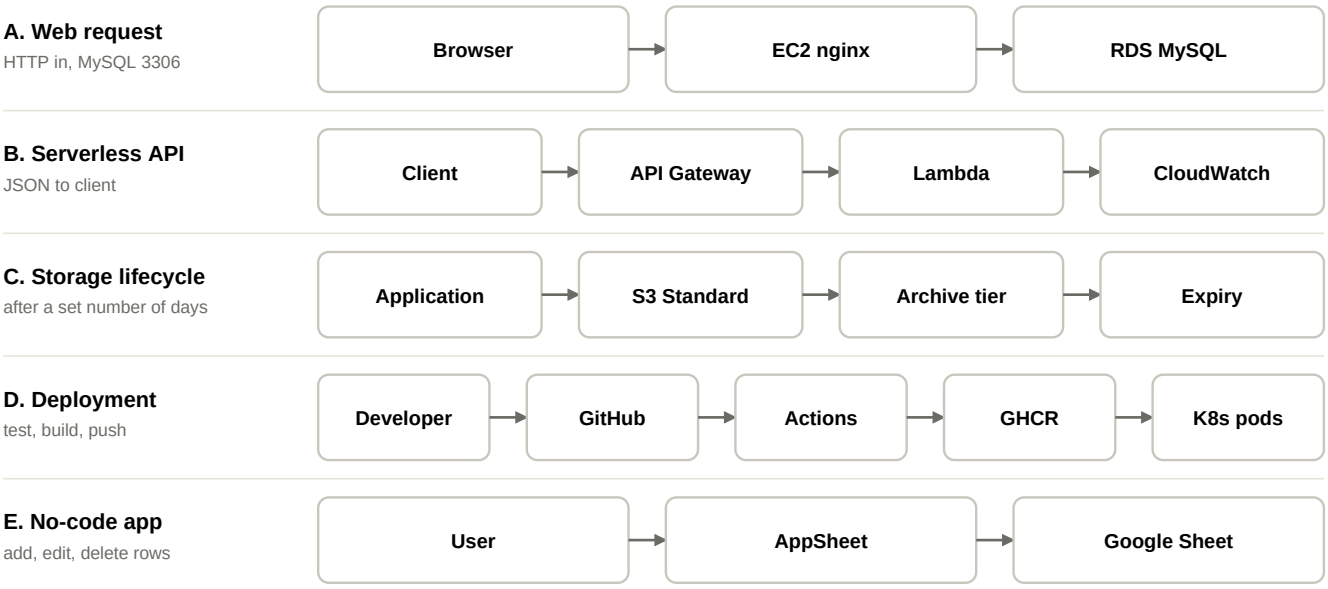
5. VPC Layout

Component	Purpose
VPC (10.0.0.0/16)	Isolated private network
Public subnet (10.0.1.0/24)	EC2 web server, routed to the Internet Gateway
Private subnets A and B (10.0.2.0/24, 10.0.3.0/24)	RDS database in two availability zones (required by the DB subnet group), no direct internet route
Internet Gateway	Internet access for the public subnet
Security group (web)	Allows inbound HTTP (80) and Docker nginx (8080). No SSH rule; access through Session Manager
Security group (db)	Allows MySQL (3306) only from the web security group

6. Microservices Architecture

Three containerised services run on Kubernetes (a local kind cluster): frontend-service, backend-service and cicd-app. Each is a Deployment with 2 replicas for availability, exposed inside the cluster through a ClusterIP Service (frontend-svc, backend-svc, cicd-app-svc). The frontend and backend were reached through port-forwarding (localhost:8081 and localhost:8082). The cicd-app image is built by the pipeline and pulled from GitHub Container Registry. Kubernetes restarts failed pods automatically and balances traffic across replicas.

7. Data Flow Diagram



data flows: 5 flows, A to E

Each row is one flow, read left to right: web traffic reaches the database, the API returns JSON, S3 data ages into archive, code reaches pods, and the no-code app writes to a sheet.

8. Security Model

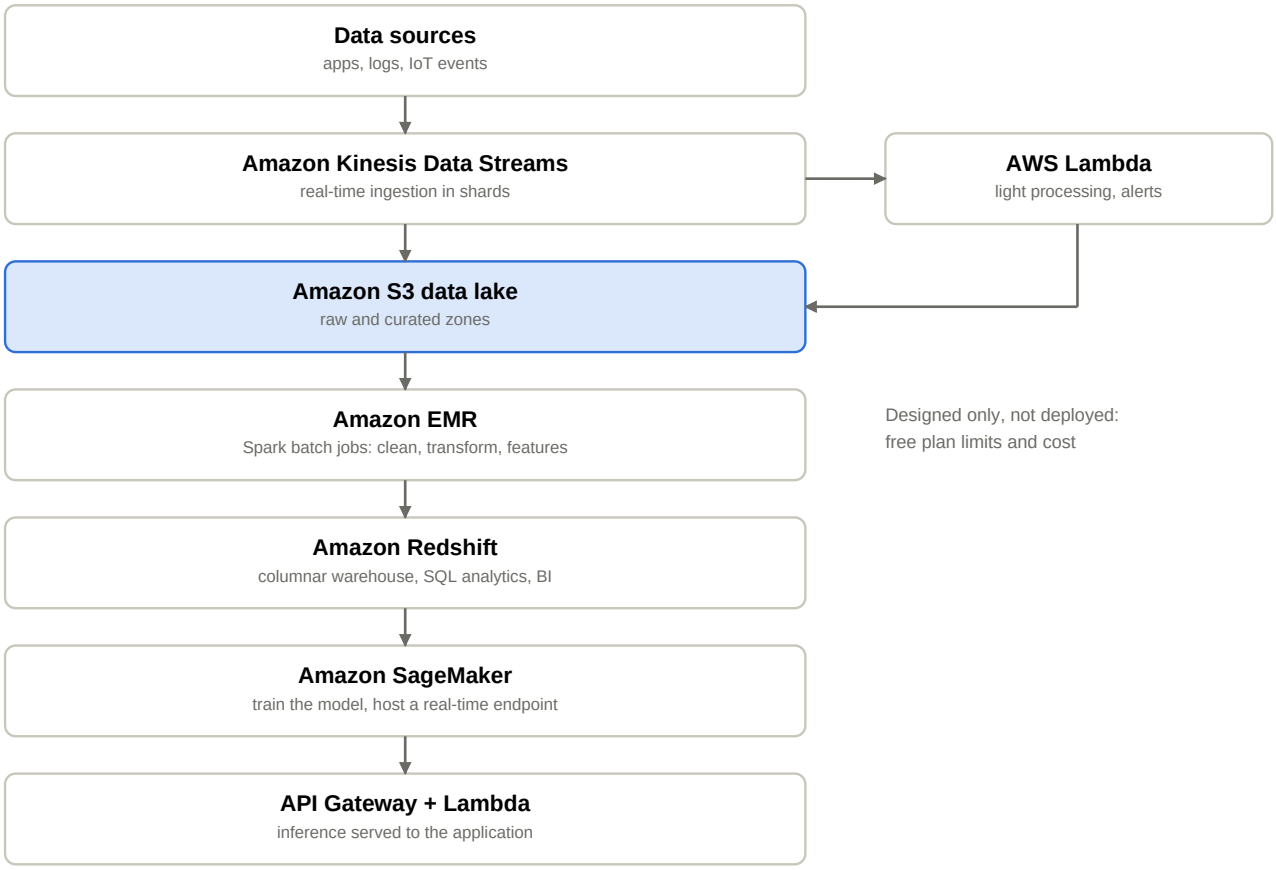
Area	Control
Identity (IAM)	Dedicated IAM user for Terraform instead of the root account; IAM role for EC2 limited to Systems Manager access; read-only IAM user; MFA recommended for the root and admin accounts
Network	Database in private subnets with no public access; DB security group accepts traffic only from the web security group; web security group has no SSH rule
Encryption at rest	S3 server-side encryption, RDS storage encryption, encrypted EC2 root volume
Encryption in transit	HTTPS for API Gateway and for the GitHub container registry; database traffic stays inside the private subnets
Data exposure	S3 block public access enabled
Secrets	RDS password generated by Terraform (random_password), never hard-coded; .tfstate and .terraform/ excluded from Git through .gitignore
CI/CD	Pipeline uses the built-in GITHUB_TOKEN with minimum permissions (contents read, packages write)
Monitoring	CloudWatch dashboard and CPU alarm with SNS email, billing budget to prevent unexpected cost

9. Infrastructure as Code and CI/CD

Terraform files main.tf and monitoring.tf define the VPC, subnets, security groups, EC2, S3 with lifecycle, RDS, IAM and CloudWatch. The Lambda function and API Gateway were created from the AWS console. The GitHub Actions workflow .github/workflows/ci-cd.yml has three jobs: test (npm test), build-and-push (Docker image to ghcr.io) and deploy (validates the Kubernetes manifest k8s-app.yaml). Because the Kubernetes cluster is local, the manifest was applied manually with kubectl apply -f k8s-app.yaml. All code is in the GitHub repository.

10. Big Data and ML Pipeline Summary (Architecture)

These services were designed but not deployed, to avoid cost and because the AWS free plan restricts some of them (for example, Kinesis returned a SubscriptionRequiredException).



big data and ML pipeline: 7 stages

Events enter through Kinesis and land in S3, where EMR prepares them and Redshift analyses them; SageMaker serves predictions through an API.

Service	Role in the pipeline
Kinesis Data Streams	Ingest real-time events (orders, clicks) in shards
S3	Central data lake, lifecycle policies for archival
EMR	Spark and Hadoop jobs for large-scale transformation
Redshift	Columnar warehouse for fast analytical queries
SageMaker	Model training, hosting and automated inference

Example use case: order events stream in through Kinesis, are stored in S3, cleaned by EMR, analysed in Redshift, and a SageMaker model predicts demand or fraud, with predictions served through an API.

11. No-Code Application

Mendix signup allows only work email addresses, so an equivalent no-code platform, Google AppSheet, was used. The app is built on a Google Sheet (cloud database) and supports add, edit and delete. A record added in the app (rohan) appeared in the sheet, proving the integration. The app runs in prototype mode, since production deployment needs a paid plan, and is reachable by URL from the browser.

12. Cost Control

A billing budget was created first. The Terraform-built AWS resources were destroyed with terraform destroy, and the Lambda function and API Gateway (created from the console) were deleted manually, after the evidence was captured. The Kubernetes cluster runs locally (kind) at no cost, instead of EKS. GitHub, GHCR and AppSheet were used on free plans.

13. Conclusion

The project covers the full cloud lifecycle: IaaS provisioning, networking and storage, serverless, Kubernetes microservices, security, monitoring, IaC, CI/CD, a no-code app and a big data and ML design. The result is a repeatable, automated and cost-aware blueprint.